

Beyond Network Size: Factors Shaping Autonomous Cyber Agent Strategy

Kenechukwu Nwodo¹[0009-0000-7208-909X], Justin Van Roy²[0009-0004-2535-7409], Dziyana Hnedzko²[0009-0009-9559-0317], and Angelos Stavrou^{1,2}[0000-0001-9888-0592]

¹ Virginia Tech, Alexandria, VA, USA

{nwodok, angelos}@vt.edu

² A2 Labs, Arlington, VA, USA

{jvanroy, dhnedzko}@a2labs.com

Abstract. Autonomous cyber agents are increasingly deployed within network testbeds to benchmark their offensive and defensive capabilities. However, a scaling fallacy exists, suggesting that increased size is a primary driver behind increased complexity in defender-attacker interactions. In this paper, we challenge this paradigm through a comprehensive evaluation of autonomous cyber agents in simulated network environments with randomized topologies. We analyze agent behaviors after varying security objectives, the number of structural chokepoints, topology sizes, and attacker/defender agent selection. Across 2,190 episodes, we find that security objectives determine how an attacker selects its actions. Privilege Escalation actions rise from 170 to 430 when the security objective pivots from Availability to Confidentiality, while Impact actions fall from 528 to 288. The chokepoint structure has a comparably significant effect on agent strategy. A single chokepoint inverts the attacker’s strategy from deep and concentrated compromise to broad and shallow compromise, though effects of a second chokepoint vary in effectiveness between defender policies of different security objectives. Among the defender policies tested, only restore-style remediation produces a measurable change in outcomes. Finally, larger networks also benefit the attacker, since compromise breadth grows faster than attack pressure or compromise depth.

Keywords: Autonomous Agents · Network Security · Chokepoints.

1 Introduction

The scale and speed of modern network attacks have pushed defenders toward automation over the past decade. Autonomous agents have thus become a standard fixture in network defense research, taking on both attacking and defending roles in simulated network environments [36]. Such agents are designed to carry out cyber operations without human intervention. Environments such as CybORG and the CAGE Challenge series have standardized the development and evaluation of these agents in enterprise networks, providing attacker, defender,

and user agent roles, as well as network scenarios against which agent policies can be trained and compared [14, 15, 32]. Heuristic and reinforcement learning (RL) methods are two predominant approaches for developing such agents, with heuristic agents relying on rule-based logic to make decisions and RL agents discovering policies through environment interaction [2, 7].

Evaluating autonomous cyber agents requires controlled environments where offensive and defensive strategies can be tested without risk to operational infrastructure. Agent performance is typically measured through reward-based metrics that reflect how well an agent achieves its objective over an episode [25], though recent work has explored richer outcome measures such as resilience [6]. These measures provide a useful summary of agent behavior, but they do not reveal which properties of the environment produced that behavior.

Current practice conflates network size with environmental complexity, typically using fixed topologies and one security objective [15], and realistic training environments remain an open gap [11, 36]. Attempts to make the problem harder have meant larger networks: CAGE-4 targets enterprise scale [14], while others evaluate across networks of 10 to 40 and 2 to 50 nodes [4, 34]. This is a scaling fallacy, since a larger network does not necessarily produce a more complex problem. Scaling CAGE-2 to 1,000 hosts makes the attacker’s task harder without making the problem more complex [31]. What is rarely varied is what shapes the interaction: security objectives, structural layouts, and agent choices.

To this end, this paper comprehensively studies the factors that shape autonomous agent strategy for network defense. We evaluate heuristic agents in an extended CybORG environment that addresses the fixed-topology limitation of the CAGE scenarios through randomized topology generation and random attacker initialization. The environment models a Student Information System (SIS), a representative enterprise network managing student records, authentication, and critical services across segmented subnets. We run agents across three security objectives (Confidentiality, Integrity, and Availability), three network sizes, and three chokepoint configurations, with multiple attacker/defender agent pairings. Each agent pairing is evaluated over multiple episodes, and we measure how each factor reshapes agent strategy using episode-level metrics that capture action composition, compromise depth and breadth, and remediation behavior.

We performed an extensive evaluation spanning 438 runs and 2,190 episodes. Beyond network size, a key finding is that the security objective determines the composition of attacker actions, not how many attacks the attacker attempts overall. Privilege Escalation actions increase from 170 to 430 from Availability to Confidentiality, while Impact actions fall from 528 to 288 over the same range. Chokepoints prove similarly conditional. A single chokepoint inverts attacker strategy in every objective, yet a second chokepoint only continues to help in Availability, and reduces effectiveness for most defender policies in Integrity and Confidentiality. Moreover, passive monitoring accounts for most defender behavior across the board, and restore-style remediation is the only policy that differs from it in a measurable way. Lastly, network size itself still favors the

attacker, with compromise breadth growing faster than either attack pressure or compromise depth as the network scales.

To summarize, the contributions of this work are:

- We leverage an extended CybORG environment with randomized topologies and random attacker initialization to evaluate agents under more realistic and varied network conditions.
- We introduce a factor-isolation evaluation framework for autonomous cyber agents that treats security objective, chokepoint count, topology size, and agent selection as controlled experimental variables.
- We examine how each environmental factor shapes agent strategy, using metrics spanning attacker pressure, compromise depth and breadth, derived strategy metrics, defender efficacy, and objective-specific counters.

2 Background

2.1 Autonomous Cyber Defense

An autonomous cyber defense problem can be framed as a sequential decision problem in which a defender selects actions over time based on incomplete information about the network. At each step, the defender receives a noisy observation derived from monitoring rather than the true network state, and must choose an action from a fixed set without knowing where an attacker is or its next move.

A common formulation of this is the partially observable Markov decision process (POMDP), in which the defender maintains a belief over the network state and updates it as observations arrive [24]. Variants of this formulation extend it to two-player games, optimal-stopping problems, and structural models such as attack graphs, but each shares the same underlying problem: an attacker moving through a network under uncertainty and a defender acting on incomplete information [9, 10, 19, 37].

This class of problem is commonly studied using the Cyber Operations Research Gym (CybORG), a simulated enterprise network of hosts, subnets, and services with a fixed set of red and blue actions [32, 33]. The CAGE Challenge series standardizes scenarios on top of CybORG and fixes a three-agent structure of red, blue, and green agents [14, 15]. Red (attacker) agents perform reconnaissance, exploitation, privilege escalation, and impact actions that degrade the targeted objective. Blue (defender) agents monitor the network, analyze suspected hosts, deploy decoys, remove malicious processes, and restore compromised hosts to a known-good state. Finally, green agents simulate benign user activity. The environment is partially observable by design, and the CAGE-2 scenario has been studied as a POMDP [20]. In this work we use an extended version of CybORG that replaces the standard fixed topology with a randomized topology generator, randomizes the red agent starting position across episodes, and extends the threat model to cover all three CIA workflow objectives. We run fixed heuristic agents inside this environment rather than learning agents, which lets us study how the environment itself shapes the strategies that emerge.

2.2 Security Objectives: The CIA Triad

Confidentiality, Integrity, and Availability are the three core security objectives [28], and can be used as workflows. Availability concerns whether a service or resource is accessible to legitimate users. Integrity concerns whether data has been modified or corrupted without authorization. Confidentiality concerns whether sensitive information has been accessed or disclosed without authorization. Most autonomous cyber defense benchmarks instrument only one of these objectives. Work built on the CAGE scenarios primarily rewards service continuity, so it evaluates availability-centered defense behavior [7, 15]. Evaluating agents against a single objective covers only one region of the attacker action space and risks overfitting to that threat type, leaving generalization across objectives untested.

The three objectives produce different attacker behaviors even when the network structure is held fixed. Each objective also engages a different part of the defender action set, meaning the defender’s optimal response changes with the objective in play. We treat the security objective as a controlled experimental factor rather than a fixed property of the scenario.

2.3 Structural Factors: Chokepoints and Topology Size

Two structural properties of a network can shape how an attacker spreads through it: the number of chokepoints [8] it contains, and the topology size. In this work, both properties serve as controlled factors in the experimental design.

A chokepoint is a host that every attack path from the entry point to the target must pass through. Removing it partitions the network and disconnects the attacker from the target, making it a minimum-cut vertex in the network graph. The practical test is straightforward: if removing a node creates an air gap between the attacker and the target, it is a chokepoint. Because every attack path passes through a chokepoint, a defender that continuously restores it can sever all routes to the target at once. This works because an attacker requires a minimum of two actions to move through a node, and a restore resets the attacker before both actions can complete. However, continuously restoring a node takes it offline, which in a real network would render any service depending on it unavailable. More chokepoints give the defender more opportunities to apply this strategy, but at the cost of reduced path redundancy.

Topology size is the second structural factor. In this work, size refers to the number of subnets and the number of hosts per subnet, and we use topology size and network size interchangeably throughout this paper to refer to this same quantity. A larger network gives the attacker more hosts to compromise, and we vary size to test whether scale alone drives the scaling fallacy.

3 Methodology

This section presents the design of our framework, developed to identify how environmental factors determine the strategies of autonomous agents in network

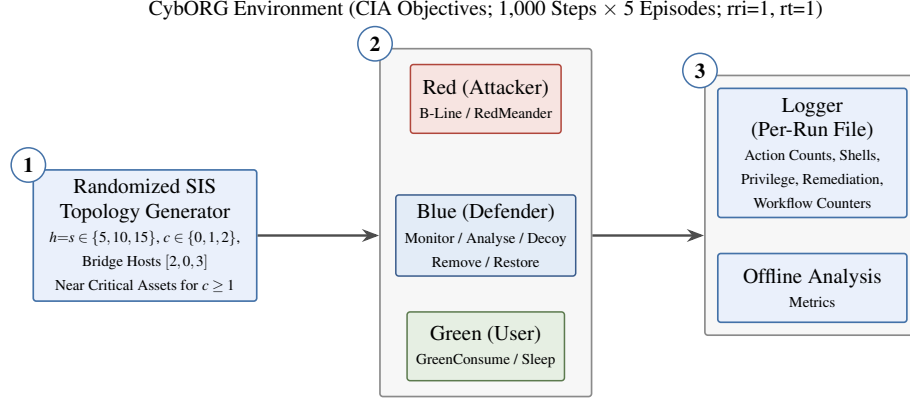


Fig. 1. System design. A randomized SIS topology generator instantiates a CybORG episode for the requested size and chokepoint configuration; red, blue, and green agents interact under one security objective for 1,000 steps; a per-episode logger records strategy-level metrics that an offline process aggregates.

defense. The framework isolates security objective, chokepoint count, topology size, and agent selection as controlled factors, using an episode, a multi-agent game between a single red attacker agent, a single blue defender agent, and a single green benign user agent, executed on a simulated Student Information System (SIS) network for 1,000 steps.

3.1 System Design

Fig. 1 shows the system comprising three stages.

Topology generation. A randomized SIS topology generator instantiates a network for the requested size (h hosts per subnet, s subnets) and chokepoint count (c per subnet), using the generator described in [29]. Subnets form a linear chain. Each subnet is connected only to the next, by linking a random subset of its hosts to a random subset of the following subnet’s hosts, so reaching the critical-asset subnet at the end of the chain requires traversing every intermediate subnet. Two parameters govern chokepoint placement. The count c is applied to every subnet, so $c=2$ places one more chokepoint per subnet than $c=1$. For $c \geq 1$, a fixed allocation of additional bridge hosts, $[2, 0, 3]$, is applied to the last three subnets before the critical-asset subnet. We hold this allocation constant across all runs so that c is the only chokepoint parameter that varies, while the hosts selected remain randomized with the rest of the topology. Chokepoints are hosts, so networks with higher c contain more hosts; network size is varied independently as a separate factor. **Fig. 4** shows an example topology.

Episode execution. The CybORG SIS environment runs the red, blue, and green agent episode for 1,000 steps, with five episodes per run. We enabled the randomization of the network topology ($rt = 1$) and of the attacker’s starting

position ($rri = 1$) independently for each episode, so that no episode repeats the same entry point on the same layout. We select the red agent from two policies, B-Line and RedMeander, the blue agent from five policies, Monitor, Analyse, Decoy, Remove, and Restore, and the green agent from two policies, GreenConsume and Sleep.

Logging and analysis. A logger writes one structured file per run, containing a record for each of its five episodes with per-agent action counts, per-host shell counts, privilege state, remediation outcomes, and objective-specific (workflow) counters. Offline analysis generates the strategy metrics reported in [Section 4](#).

3.2 Security Objectives

We use an extended CybORG that models three security objectives, one per element of the CIA triad, sharing an identical topology generator and red, blue, and green action sets outside the objective-specific additions detailed below.

Availability. Measures legitimate service continuity. A green `ConsumeService` action exercises services, and the defender’s `Restore` action returns hosts to a known-good state. The attacker’s `Impact` action is the mechanism that degrades these services once red reaches a host it can affect.

Integrity. Extends Availability with a corrupted-traffic channel, in which red can poison green requests so that green observes attacker-controlled responses. Tampering is reported independently as `corrupted_access_green_total`, the only mechanism in our system that measures tampering separately from compromise counts. The blue action set adds `CheckForCorruption` and `RestoreContent`.

Confidentiality. Replaces the Availability `StopService` dispatch with `DataExfiltration`, so `Impact` succeeds only when the target host’s availability value is `Medium` or `High`, causing brute-forced `Impact` attempts against `Low` or `None` hosts to fail with near-certainty. The blue action set adds `Block`, which sets a `traffic_blocked` flag on a host and causes any subsequent `DataExfiltration` attempt to fail.

3.3 Agent Policies

Each agent uses one of the following policies. We use the heuristic policies provided with the extended CybORG environment described in [\[29\]](#), which is available at [\[1\]](#).

Red. *B-Line* follows a sequential kill chain, discovering and exploiting hosts as they become reachable and targeting critical-asset hosts as soon as possible. It is more direct and aggressive than *Meander*, but builds its path from observations rather than prior knowledge of the network. *Meander* explores reachable hosts broadly before committing to impact, rather than making straight for the target. The two differ in how they choose their next action. *B-Line* works through a fixed sequence of stages, discovering, scanning, exploiting, escalating, and impacting, advancing only as each stage succeeds and retrying earlier stages when one fails. *Meander* instead re-evaluates the same ordering of action types at each step,

working across the hosts it has already observed. Both choose at random among the hosts available to them, B-Line within its current subnet and Meander across everything it has seen. Neither uses any information about the topology beyond the hosts and subnets in its current observation.

Blue. *Monitor* always performs the **Monitor** action and takes no remedial action. *Analyse* monitors, then runs the **Analyse** action on flagged hosts, gathering information without remediating. *Decoy* proactively deploys all available decoys at the start of the episode, prioritizing bridge and critical-asset-adjacent hosts, then falls back to the **Monitor** action once decoys are exhausted. *Remove* monitors, then runs the **Remove** action on flagged hosts; it is limited to user-level shells and cannot clear root access. *Restore* monitors, then runs the **Restore** action on flagged hosts, clearing both user and root shells, and behaves identically across all three objectives. A host becomes flagged when the **Monitor** action reports a running process on it, and *Analyse*, *Remove*, and *Restore* perform **Monitor** on any step where no host is flagged.

Green. *Consume* issues **GreenConsumeService** calls generating the traffic. *Sleep* takes no action, isolating the red and blue interaction from benign traffic.

3.4 Experimental Factors

A run combines one level of each factor introduced above: security objective, red agent, blue agent, green agent, network size, and chokepoint count. The Confidentiality objective is evaluated with only **Sleep** by design. Each run executes five episodes of 1,000 steps. The actual design comprises 438 runs and 2,190 episodes: 180 runs (900 episodes) each for Availability and Integrity, and 78 runs (390 episodes) for Confidentiality.

3.5 Metrics

Each episode yields five metrics: attacker pressure, compromise depth and breadth, derived strategy metrics, defender efficacy, and objective-specific counters.

Attacker Pressure. We track **tp_attacks** and **fp_attacks**, classified by the environment’s logger, where a false positive is a successful exploit against a host with an active defender decoy. This distinguishes exploitation directed at genuine targets from exploitation the defender has successfully misdirected. We also track reconnaissance volume and per-category action totals for **Impact**, **PrivilegeEscalate**, **Exploit**, and **Discover/Scan** actions.

Compromise Depth and Breadth. We measure breadth with **unique_compromises**, a cumulative count of distinct hosts rooted, versus **total_compromised_hosts**, a snapshot of hosts with an active shell at episode’s end that can fall once the defender restores a host. Depth is captured by **cumulative_infected_hosts**, which counts every compromise event including re-entries after a restore, together with per-host shell counts and root shells per compromised host.

Derived Strategy Metrics. We define a depth-breadth ratio

$$\rho = \frac{\text{cumulative_infected_hosts}}{\text{unique_compromises}}, \quad (1)$$

as our primary scalar summary of attacker strategy ($\rho \gg 1$ indicates deep, concentrated compromise; $\rho \approx 1$ indicates broad, distributed compromise), and we report it as the mean over episodes of the per-episode ratio. Episodes in which the attacker compromises no host are excluded from this average, since ρ is undefined when `unique_compromises` is zero. We also define re-compromise events as cumulative minus unique infections, which exposes restore-induced re-compromise loops. Both quantities are computed identically across every objective, agent pairing, and topology configuration, which makes them comparable across the factors we vary.

Defender Efficacy. We measure true and false positive remove and restore counts, classified by the same logging mechanism, where a false positive denotes a defensive action taken against an already-clean host, along with decoy deployments and conditional restores against hosts with root-level access or with more than one active shell. This distinguishes genuine remediation from wasted defensive effort. The conditional restore counts single out restores against hosts with root-level access or multiple simultaneous shells, both signs of more entrenched compromise.

Objective-Specific Counters. We track green consume successes and failures for Availability, corrupted green accesses for Integrity, and block actions for Confidentiality. Unlike the metrics above, each counter is tied to a mechanism that exists in only one objective. Outside its own objective, a counter is structurally zero rather than merely inactive, since the underlying mechanism is absent from the environment entirely.

Table 1 defines the terms and abbreviations used throughout the paper.

Table 1. Definition table for terms and abbreviations used throughout this paper.

Term	Definition	Term	Definition
Objective	Security objective an episode runs under: Confidentiality, Integrity, or Availability. Also called a workflow.	True positive (TP)	Successful exploit on a genuine host, or remediation of a genuinely compromised host.
Episode, N	A 1,000-step game between one red, one blue, and one green agent. N counts episodes.	False positive (FP)	Successful exploit on a host with an active decoy, or remediation of an already-clean host.
Run	One combination of every experimental factor, run as five episodes.	Breadth (unique compromises)	Distinct hosts rooted over the episode.
Chokepoint, c	A host every attack path to the target must pass through, so removing it partitions the network. c counts them per subnet.	Depth (cumulative infections)	Every compromise event, including re-entry after a restore.
h, s	Hosts per subnet and number of subnets. $h = s$ throughout.	Root shells	Root-level shells held on a compromised host.
Penetration rate	Fraction of the network the attacker compromises.	Re-compromise events	Cumulative infections minus unique compromises.
		ρ	Depth-breadth ratio (Equation (1)). $\rho \gg 1$ is deep and concentrated; $\rho \approx 1$ is broad and distributed.

4 Evaluation

We implement the agent evaluation for the simulated environment in Python 3.10.12. Our experiments run on an Ubuntu 22.04.5 server with 16 CPUs (QEMU Virtual CPU version 2.5+, x86) and 64 GB of RAM.

We evaluate four factors hypothesized to influence defender-attacker interactions in the SIS environment: (i) security objective, (ii) chokepoint density, (iii) topology size, and (iv) agent selection on each side. All results are drawn from the 2,190 episodes described in [Section 3.4](#). Unless otherwise noted, values are mean $\pm$ standard error (SE) over all episodes in the relevant configuration, with N reported in each table.

Where a factor changes agent behavior, we report the corresponding effect on outcomes alongside it. Root shells measure how much of the network the attacker ends up holding, and remediation counts measure what the defender spends to contain it; both are quantities penalized by the reward functions used in CybORG-based evaluations [\[6, 15\]](#), reported here as separate counts rather than as a single reward score.

4.1 Effect of the Security Objective

Table 2. Security objective effect on attacker pressure and action composition. All configurations are combined. Values are mean $\pm$ SE per episode.

Metric	Availability	Integrity	Confidentiality
TP attacks / episode	169.8 $\pm$ 5.39	170.5 $\pm$ 5.30	180.7 $\pm$ 7.70
Corrupted green accesses	0.00 $\pm$ 0.00	972.0 $\pm$ 61.26	0.00 $\pm$ 0.00
Privilege escalation	169.8 $\pm$ 5.39	221.2 $\pm$ 5.97	430.4 $\pm$ 13.91
Remote service exploitation	169.8 $\pm$ 5.39	170.5 $\pm$ 5.30	180.7 $\pm$ 7.70
Impact actions	528.3 $\pm$ 14.25	470.9 $\pm$ 13.40	288.0 $\pm$ 14.87

[Table 2](#) shows near-identical attacker pressure across objectives. TP attacks per episode are 170 (Availability), 170 (Integrity), and 181 (Confidentiality), and in every objective this figure is identical to the number of `ExploitRemoteService` actions, since TP attacks in this table count successful exploitation specifically rather than total offensive actions. By construction, Availability and Confidentiality see zero corrupted green traffic, while Integrity exhibits 972 corrupted green accesses per episode, the only mechanism that measures tampering independently of compromise counts.

What changes across objectives is where that effort goes. The number of `PrivilegeEscalate` actions increases from 170 (Availability) to 221 (Integrity) to 430 (Confidentiality), while the number of `Impact` actions decreases in the opposite

direction, from 528 to 288, a 45% drop from Availability to Confidentiality. **Impact** outnumbers **PrivilegeEscalate** by roughly $3\times$ in Availability and $2\times$ in Integrity, but this reverses in Confidentiality, where **PrivilegeEscalate** overtakes **Impact**. Under Confidentiality, **Impact** only succeeds against hosts with medium or high availability value, so brute-forced **Impact** attempts against low- or no-availability hosts fail. **PrivilegeEscalate** counts rise over the same range that **Impact** counts fall.

Takeaway. The security objective shifts effort between **PrivilegeEscalate** and **Impact** without changing the total, which sums to 698, 692, and 718. A benchmark reporting only that total would miss the shift.

4.2 Chokepoint Effect

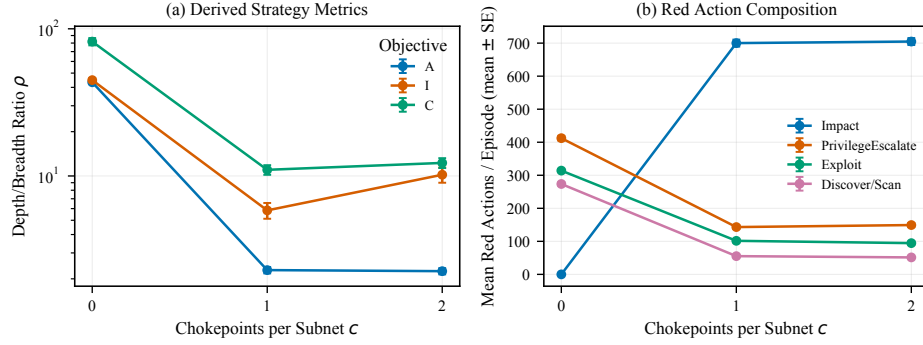


Fig. 2. Chokepoints invert attacker strategy. (a) The derived depth/breadth ratio collapses by an order of magnitude from $c=0$ to $c=1$, holding at $c=2$ except in Integrity (legend: A = Availability, I = Integrity, C = Confidentiality); (b) the red action mix, with combined CIA objectives, shifts from **PrivilegeEscalate** to **Impact**.

Chokepoints fundamentally alter the attacker’s strategy, as seen in Fig. 2 and Table 3. At $c = 0$ the attacker operates deep and concentrated, compromising ≈ 10 hosts regardless of objective. Root shells per host are 24 (Availability) and 25 (Integrity), but only 11 in Confidentiality, and **Impact** actions are 0 everywhere, with effort instead consumed by **PrivilegeEscalate** (353 in Availability, 663 in Confidentiality).

Introducing a single chokepoint ($c = 1$) inverts this to broad and shallow. Breadth grows to 44 (Availability), 46 (Integrity), and 55 (Confidentiality) hosts, while depth collapses to ≈ 1 root shell per host, and **Impact** spikes to 783 (Availability), 709 (Integrity), and 469 (Confidentiality). Neither red policy uses information about the topology, so the attacker does not respond to the chokepoint directly. The policies are unchanged across chokepoint levels, so this shift comes from the network structure rather than from the attacker.

Table 3. Chokepoint effect on attacker strategy per security objective. With chokepoints enabled the attacker shifts from deep localized exploitation (high shells/host, many **PrivilegeEscalate** actions) to broad, shallow disruption (more hosts, **Impact**-heavy). Values are mean $\pm$ SE.

Chokepoints	Objective	N	Breadth (Hosts)	Depth (Shells/Host)	Impact Actions	Privilege Escalate	Exploit Actions
0	Availability	300	9.53 $\pm$ 0.25	24.35 $\pm$ 0.98	0.00 $\pm$ 0.00	352.6 $\pm$ 3.17	352.6 $\pm$ 3.17
0	Integrity	300	9.45 $\pm$ 0.24	25.20 $\pm$ 1.07	0.00 $\pm$ 0.00	346.6 $\pm$ 3.51	346.8 $\pm$ 3.51
0	Confidentiality	150	9.63 $\pm$ 0.33	11.15 $\pm$ 1.16	0.00 $\pm$ 0.00	663.3 $\pm$ 25.32	171.8 $\pm$ 13.06
1	Availability	300	44.01 $\pm$ 2.54	1.01 $\pm$ 0.00	783.1 $\pm$ 14.78	82.05 $\pm$ 6.55	82.08 $\pm$ 6.56
1	Integrity	300	45.56 $\pm$ 2.65	1.01 $\pm$ 0.00	709.4 $\pm$ 16.02	148.8 $\pm$ 10.23	86.92 $\pm$ 6.63
1	Confidentiality	120	55.42 $\pm$ 4.86	1.57 $\pm$ 0.19	469.2 $\pm$ 21.73	281.9 $\pm$ 8.04	186.9 $\pm$ 13.26
2	Availability	300	39.77 $\pm$ 2.48	1.02 $\pm$ 0.00	801.6 $\pm$ 14.57	74.85 $\pm$ 6.45	74.83 $\pm$ 6.45
2	Integrity	300	40.95 $\pm$ 2.51	1.01 $\pm$ 0.00	703.2 $\pm$ 15.87	168.3 $\pm$ 11.18	77.89 $\pm$ 6.39
2	Confidentiality	120	52.77 $\pm$ 4.87	1.56 $\pm$ 0.18	466.6 $\pm$ 21.46	287.8 $\pm$ 7.83	185.6 $\pm$ 13.63

The depth/breadth ratio ρ (Equation (1)) drops by an order of magnitude at every objective from $c = 0$ to $c \geq 1$ (Availability 43 to 2, Integrity 45 to 6, Confidentiality 82 to 11; Table 6). The ratio stays almost constant from $c = 1$ to $c = 2$ in Availability and Confidentiality, but nearly doubles in Integrity.

From the defender’s perspective chokepoints are substantially beneficial (full detail in Table 7). For three of the four non-Restore policies in Availability, one chokepoint cuts cumulative infections from 329 to ≈ 59 per episode. *Remove* sees a smaller drop, to 68. This pattern of continued improvement from $c = 1$ to $c = 2$ holds only in Availability. In Integrity, three of the four non-Restore policies see cumulative infections rise rather than fall at the second chokepoint, from 124 to 171 for *Monitor* and 127 to 155 for *Analyse*. Confidentiality shows the same pattern, 254 to 256 for *Monitor* and 263 to 284 for *Remove*. The improvement described above is, therefore, an Availability-specific effect, not a general property of the mechanism. For *Restore*, the effect is the mirror image of its re-compromise pattern. Its cumulative count falls from 466 at $c = 0$ to 225 at $c = 1$, because the chokepoint suppresses the exploit-restore-re-exploit loop that the policy otherwise sustains. This is the behavioral signature of the min-cut mechanism of chokepoints, where the attacker abandons deep single-target progression and spreads across whatever remains reachable.

Takeaway. A single chokepoint inverts attacker strategy and cuts cumulative infections in every objective. A second continues to help in Availability, but most policies worsen under Integrity and Confidentiality.

4.3 Defender and Attacker Selection

Table 4 shows *Monitor*, *Analyse*, *Decoy*, and *Remove* producing nearly identical root shells (105–114) and cumulative infections (213–225), combined across all three objectives. *Restore* stands apart with 25 root shells but 357 cumulative

Table 4. Defender policy comparison combined across all three security objectives (Availability, Integrity, Confidentiality) and network configurations. Values are mean $\pm$ SE per episode. Remove (TP) and Restore (TP) are (23.46 ± 1.60) and (252.4 ± 7.44) . Note that *Analyse* has 390 episodes since its Confidentiality runs cover only $c = 0$.

Blue policy	N	Active compromise (root shells)	Cumulative infections
BlueMonitor	450	105.0 ± 4.72	218.7 ± 9.65
ReactAnalyse	390	112.7 ± 5.19	212.5 ± 11.00
ReactDecoy	450	105.4 ± 4.71	213.7 ± 9.57
ReactRemove	450	113.9 ± 4.74	225.4 ± 9.51
ReactRestore	450	25.20 ± 1.14	357.2 ± 9.59

Table 5. Attacker (red) effort and success per security objective. *Meander* outperforms *B-Line* on TP attacks primarily in Confidentiality; the two also differ in search behavior, with *B-Line* discovering roughly twice as many hosts as it scans and *Meander* scanning nearly every host it discovers. Values are mean $\pm$ SE.

Red	Objective	N	TP Attacks	Hosts Disc.	Scans	Unique Compr.	Root Shells
B-Line	Availability	450	161.9 ± 7.93	47.90 ± 2.62	21.94 ± 1.14	22.93 ± 1.14	87.66 ± 4.83
RedMeander	Availability	450	177.7 ± 7.30	45.03 ± 2.46	45.03 ± 2.46	40.43 ± 2.25	101.9 ± 4.53
B-Line	Integrity	450	160.3 ± 7.70	46.91 ± 2.54	22.64 ± 1.17	23.48 ± 1.17	87.65 ± 4.80
RedMeander	Integrity	450	180.7 ± 7.25	46.41 ± 2.48	46.38 ± 2.48	41.61 ± 2.33	101.8 ± 4.44
B-Line	Confidentiality	195	61.86 ± 7.41	52.55 ± 4.56	25.81 ± 1.91	26.81 ± 1.91	27.35 ± 1.92
RedMeander	Confidentiality	195	299.5 ± 6.11	49.13 ± 4.27	48.79 ± 4.25	48.62 ± 4.25	129.8 ± 6.48

infections, roughly a quarter the active compromise of the other four at the cost of a higher cumulative count. *Remove* is the only one of the four inactive-outcome policies that takes any remediation action at all, 23 TP removes per episode combined across objectives, but this has no visible effect on root shells or cumulative infections relative to the three policies that never act. *Restore* is likewise the only policy with any TP restores, 252 per episode combined, since every other policy restores zero hosts by design.

Broken down by objective, the non-*Restore* policies no longer track together in Confidentiality. *Monitor* and *Decoy* fall to ≈ 80 root shells, below their Availability and Integrity values, while *Remove* stays at ≈ 116 , roughly 45% higher (Table 7). *Analyse* is excluded from this Confidentiality comparison, since it is observed there only at $c = 0$ and so is not averaged over the same chokepoint levels as the other three.

Restore’s root shells stay low in every objective (25 in Availability, 24 in Integrity, 27 in Confidentiality), consistently below all four other policies regardless of objective. Additionally, *Monitor* accounts for 89% of all blue actions in Availability, 89% in Integrity, and 90% in Confidentiality, with *Restore* a distant second at $\approx 5\%$ in every objective. *Analyse* outranks *Remove* in Availability and

Integrity. Decoy deployments and Block invocations are zero across this test. Block is structurally zero outside Confidentiality and unselected by the policies evaluated, and no decoy deployment was recorded.

On the attacker side, [Table 5](#) shows agent choice affecting outcomes principally where objective-specific mechanics are activated. In Confidentiality, *Meander* achieves 300 TP attacks versus *B-Line*’s 62, a $\approx 5\times$ gap, while in Availability/Integrity the gap is only $\approx 1.1\times$ (178 vs. 162 and 181 vs. 160). Under Confidentiality, *Impact* only succeeds against hosts with medium or high availability value. *Meander*’s randomized action selection adapts to this restriction more effectively than *B-Line*’s fixed kill chain, which stalls against it. The action-type breakdown across the whole corpus tells a consistent story. *Impact* is the single most common red action in Availability (26% of all red actions) and Integrity (24%), with *ExploitRemoteService* and *PrivilegeEscalate* together accounting for a further 17–20%. In Confidentiality this ordering flips. *PrivilegeEscalate* becomes the leading single action at 22%, ahead of *Impact* at 14% and *ExploitRemoteService* at 9%.

The two red policies also differ in how they search, a behavioral trait that holds regardless of objective even though the TP-attack gap discussed above is objective-specific. *B-Line* discovers roughly twice as many hosts as it scans in every objective (48 discovered versus 22 scanned in Availability, 47 versus 23 in Integrity, 53 versus 26 in Confidentiality), consistent with its sequential, step-by-step approach, in which discovery outpaces the narrower scanning step that precedes exploitation. *Meander*, by contrast, scans nearly every host it discovers in every objective, reflecting a broader, less sequential exploration strategy.

Takeaway. Only restore-style remediation changes outcomes, cutting root shells to a quarter while increasing cumulative infections through re-compromise. The red policies differ by $5\times$ under Confidentiality but $1.1\times$ elsewhere.

4.4 Effect of Topology Size

As shown in [Fig. 3](#), increasing network size leads to consistent growth in attacker success across all outcome measures; full per-objective detail is displayed in [Table 8](#). From a network of 5 hosts per subnet and 5 subnets to 15 hosts per subnet and 15 subnets, TP attacks grow from 124 to 233 in Availability, from 127 to 207 in Integrity, and from 145 to 225 in Confidentiality.

Unique compromises grow faster than TP attacks in every objective, from 7 to 69 in Availability, from 10 to 57 in Integrity, and from 10 to 76 in Confidentiality. Root shells grow more moderately across the same range, from 71 to 132 in Availability, from 71 to 119 in Integrity, and from 52 to 114 in Confidentiality.

Defensive responses also scale, but unevenly across objectives, per [Table 8](#). TP restores grow steadily in Availability (27, 47, 76 at the three network sizes) and Confidentiality (45, 59, 80). In Integrity (30, 59, 61), the count grows only up to the middle network size and plateaus thereafter. That the restore counts go up does not mean the defender is relatively better off at larger sizes. Integrity

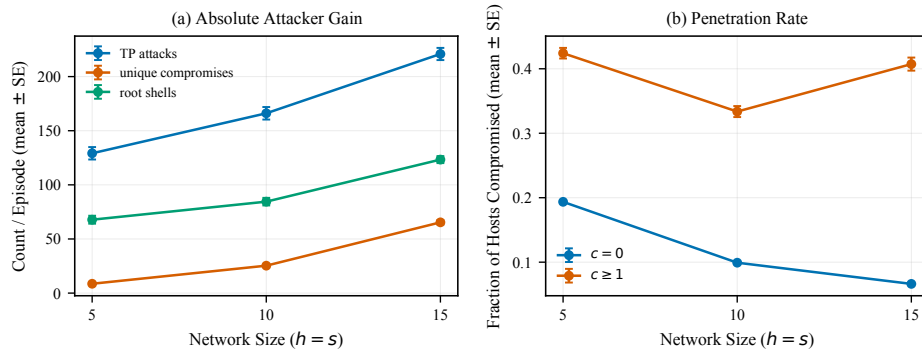


Fig. 3. Scaling benefits the attacker. (a) TP attacks, unique compromises, and root shells grow monotonically with network size; (b) the penetration rate falls with size at $c=0$, where attacker breadth is limited by effort rather than opportunity, but does not decline monotonically for $c \geq 1$.

also diverges at the second chokepoint, so two independently varied factors point to the same objective rather than to either alone.

A separate metric, penetration rate, the fraction of the network the attacker compromises, addresses that question directly. Fig. 3(b) shows penetration falling steadily with network size when no chokepoint is present, from $\approx 20\%$ to $\approx 10\%$ to $\approx 7\%$ across the three sizes, even though the attacker’s absolute reach keeps growing. Once at least one chokepoint is present, the same figure shows a different pattern rather than a flat one: penetration starts near $\approx 42\%$ at the smallest size, dips to $\approx 33\%$ at the middle size, then recovers to $\approx 41\%$ at the largest size. This is consistent with the scaling fallacy mentioned earlier. A falling penetration percentage without a chokepoint does not mean the defender is doing better at scale, only that the network is growing faster than the attacker’s reach.

Takeaway. As network size grows, breadth grows faster than attack pressure or depth, so raw compromise counts are not comparable across sizes without normalization.

5 Discussion

Generality of Findings. Two of our findings follow from the environment rather than from the agents. Under Confidentiality, Impact succeeds only against hosts of medium or high availability value, so the shift in action composition is a consequence of the objective’s success conditions. The chokepoint inversion responds to a structural constraint that neither red policy can perceive. Neither mechanism depends on how an agent selects actions, so both extend to other autonomous agents, including learned ones. The other two findings depend on the policy set. Three of the five policies never remediate and a fourth is limited to user-level shells, so the result that only restore-style remediation differs from

passive monitoring describes this baseline set. Meander’s advantage over B-Line under Confidentiality reflects how each policy selects actions, not a general difference in attacker capability.

Beyond the Tested Parameters. Future work should extend this design beyond $c \in \{0, 1, 2\}$ to determine whether the reversal seen at the second chokepoint persists, reverses again, or stabilizes at higher chokepoint counts. A similar limit applies to the episode length. Every episode runs for the same fixed number of steps regardless of objective, which may itself limit how much the `PrivilegeEscalate` and `Impact` totals can move, and varying the step size directly could separate a genuine strategic redistribution from an effect of the fixed step count.

6 Limitations

Simulation-Only Evaluation. All results in this study come from a single simulator built on CybORG. Although the simulator models user behavior and network conditions, it does so at a level of abstraction below a real or emulated deployment, and whether the structural effects reported here, chokepoint response, action redistribution driven by objective, hold once execution moves to a higher fidelity setting remains untested. Prior work has begun addressing this gap by pairing simulated and emulated environments directly [12, 21]. A high-fidelity emulated environment for this SIS setting would let future work verify whether the effects reported here, driven by objective and topology, actually transfer, rather than assuming they do.

Heuristic vs Learned Agents. All red and blue policies tested here are fixed heuristics rather than learned agents. The structural effects reported, how security objective, chokepoint density, and topology size reshape strategy, characterize how these specific heuristic policies respond to structure, not how a trained policy would. A learned (RL-based) agent, particularly one trained with reward shaping around a specific security objective or topology, may respond to the same structural factors differently than the fixed policies tested here, and some effects reported may not transfer. These policies also leave part of the defender action set unexercised, placing deception and traffic control outside the scope of our results. Future work should evaluate learned agents under this same factor isolation design to test whether these effects hold.

Single Security Objective Design. Each episode in this study is evaluated against a single security objective, Availability, Integrity, or Confidentiality, but not a combination. Real network defense frequently requires balancing multiple objectives simultaneously, protecting availability while also preserving confidentiality, for instance, and the interactions between objectives when combined are not captured by this design. The action redistribution finding discussed above may hold differently when an agent must satisfy more than one objective at once. Future work should extend this factor design to mixed-objective configurations to test these single-objective effects.

Enterprise-Only Network Scope. The network topology used throughout, a Student Information System, is representative of enterprise environments. Operational technology networks, industrial control systems, SCADA, and similar environments differ structurally in ways this topology does not capture, including real time constraints, safety constraints, legacy protocols, and different failure modes under compromise. The chokepoint and topology size effects reported here are specific to this class of network, and whether they hold, or take a different form, in other network topologies is untested. Future work should apply this same evaluation to OT or IT/OT hybrid topologies to test whether these structural effects generalize beyond enterprise networks.

7 Related Work

7.1 Simulating and Evaluating Autonomous Cyber Defense

Reinforcement learning has been shown to support autonomous network defense in adversarial settings [7], and foundational experiments with tabular Q-learning revealed both the promise and limits of applying RL to bridge automated and autonomous cyber response [2]. The CybORG environment [32] and the CAGE Challenge series [15] built on top of it provide the structure for red/blue/green agents and the action sets that we adopt and extend. Related simulators include CyberBattleSim [23], which focuses on lateral movement in fixed-topology networks, and has been extended with defender agents [16], as well as PrimAITE, a highly configurable environment supporting multiple agents that simulates realistic network traffic at the protocol level [3]. Surveys of multi-agent RL (MARL) for automated cyber defense underscore the need for sophisticated simulation and emulation environments to close the gap between training and real-world deployment [18]. Evaluation methodology itself remains an open problem in this space, as researchers in [5] evaluate defender generalization to previously unseen adversarial strategies, while a recent systematization of knowledge introduces a rubric-driven framework aligning agentic capability with the security lifecycle [13]. Separately, CAGE Challenge 4 addresses environment scale rather than evaluation methodology, introducing a scalable MARL gym targeting larger enterprise networks [14]. Our work adopts the CAGE Challenge 2 CybORG simulator environment for enterprise networks, but departs from its fixed topology by randomizing network structure across controlled experimental conditions.

7.2 Topology and Structural Effects in Networks

The influence of network structure on security is well established, spanning evolutionary game-theoretic models of attack and defense interaction [26], network risk analysis on simulated enterprise networks [17], and non-cooperative network games formulated for Information Technology (IT)/ Operational Technology (OT) networks [30]. Topology also informs defensive resource allocation more broadly than chokepoints alone. Betweenness centrality is used to place defenses

against topology inference adversaries [38] and in networked control systems [27]. Our chokepoint notion builds on the minimum-cut formulation, studied through max-flow/min-cut duality. Authors in [8] harden select chokepoint hosts to bound attacker propagation along existing communication paths, while practitioners in [35] treat identity servers, login portals, network gateways, and jump hosts as chokepoints whose remediation disrupts many attack chains at once. [22] empirically evaluates LLM-driven defender agents placed at a single topological bottleneck against autonomous attackers in dynamic cyber ranges. In this work, we measure how autonomous attacker and defender agents exploit topological chokepoints and network size under controlled variation, isolating each as an experimental variable and reporting their effect on agent behavior.

8 Conclusion

This paper comprehensively studies how network structure, security objectives, and autonomous agent selection shape agent strategy for network defense. Using a modified CybORG environment for enterprise networks, we evaluated 2,190 simulated episodes across three security objectives, three chokepoint counts, three network sizes, and multiple attacker/defender agent pairings, testing the scaling fallacy that network size alone determines the difficulty of defender-attacker interactions. The security objective changes the composition of attacker actions rather than their total volume. Privilege Escalation actions increase from 170 to 430 from Availability to Confidentiality, while Impact actions fall from 528 to 288 over the same security objectives. The two actions overtake each other only in Confidentiality. Furthermore, a single chokepoint inverts attacker strategy in every objective, from deep and concentrated compromise to broad and shallow compromise, but the effect of a second chokepoint is not consistent across objectives. This same pattern reappears when network size is varied instead of chokepoint count, suggesting that Integrity behaves differently from the other two objectives and warrants further study on its own. In terms of defender policy, only restore-style remediation performs differently from taking no action at all. Larger networks also benefit the attacker, since compromise breadth grows faster than attack pressure or compromise depth. In summary, this paper offers a factor-isolation evaluation framework for cyber agents, providing a clearer understanding of agent behaviors.

Acknowledgments. We thank the anonymous reviewers and our shepherd for their valuable feedback. This work was supported in part by the Defense Advanced Research Projects Agency (DARPA) under Contract No. HR001124C0424, and funded in part by ARL DEVCOM through the Center of Excellence on Integrated Sensing and Cyber (W911NF2320228). Any opinions are those of the authors and do not reflect any views of the funding agencies.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

A Topology Structure

Fig. 4 illustrates the structure produced by the topology generation described in Section 3.1 and an example placement of chokepoint hosts along the path to the critical-asset subnet.

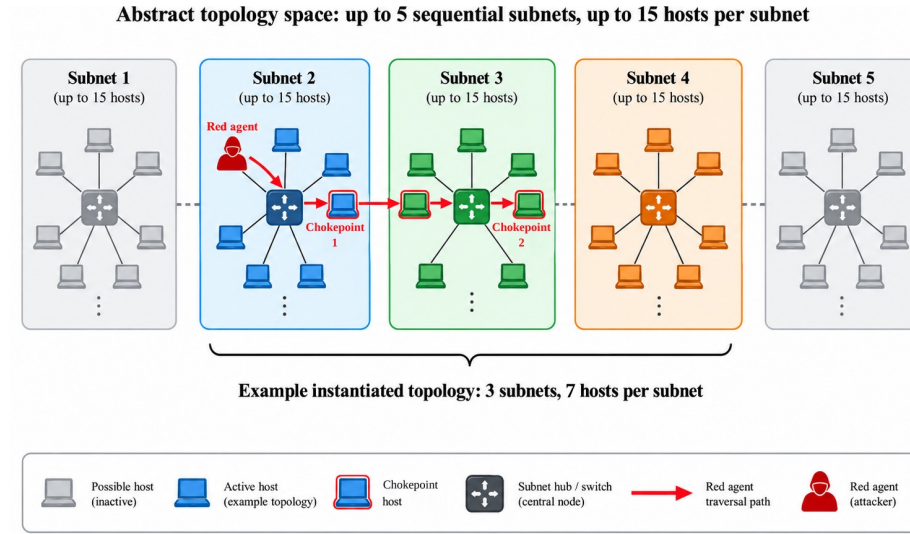


Fig. 4. Example topology. Subnets form a linear chain, with chokepoint hosts on the path toward the critical-asset subnet.

B Supplementary Results

B.1 Depth and Breadth by Chokepoint Level

Table 6 reports the full per-objective breakdown underlying the depth/breadth ratio discussed in Section 4.2, including breadth, cumulative depth, and re-compromise events at each chokepoint level.

B.2 Chokepoint Effect Detail by Defender Policy

Table 7 reports the full breakdown of blue agent performance across all three chokepoint levels and all three security objectives, including root shells, remediation counts, underlying the defender-benefit discussion in Section 4.2.

B.3 Network Topology Size Effects: Full Detail

Table 8 reports the full per-objective breakdown of metrics across all three network sizes, including root shells and the $h=10$ midpoint not discussed in Section 4.4.

Table 6. Attacker depth versus breadth by chokepoint level. The depth/breadth ratio (ρ , cumulative infections per unique compromised host) is the primary derived strategy metric; re-compromise events are cumulative minus unique infections. Values are mean $\pm$ SE.

Objective	Chokepoints	N	Breadth (Unique Hosts)	Depth (Cum. Infections)	Depth/Breadth Ratio	Re-Compromise Events
Availability	0	300	10.00 $\pm$ 0.24	356.5 $\pm$ 3.21	43.32 $\pm$ 1.20	346.5 $\pm$ 3.20
Availability	1	300	44.61 $\pm$ 2.54	93.76 $\pm$ 6.81	2.29 $\pm$ 0.12	49.15 $\pm$ 5.68
Availability	2	300	40.42 $\pm$ 2.49	85.48 $\pm$ 6.69	2.25 $\pm$ 0.11	45.06 $\pm$ 5.54
Integrity	0	300	9.73 $\pm$ 0.24	350.6 $\pm$ 3.56	44.81 $\pm$ 1.36	340.9 $\pm$ 3.53
Integrity	1	300	46.22 $\pm$ 2.65	160.3 $\pm$ 10.31	5.84 $\pm$ 0.72	114.1 $\pm$ 9.99
Integrity	2	300	41.68 $\pm$ 2.52	179.0 $\pm$ 11.17	10.20 $\pm$ 1.20	137.3 $\pm$ 11.16
Confidentiality	0	150	10.00 $\pm$ 0.33	667.0 $\pm$ 25.26	81.84 $\pm$ 4.68	657.0 $\pm$ 25.27
Confidentiality	1	120	56.04 $\pm$ 4.86	297.2 $\pm$ 8.38	11.01 $\pm$ 0.83	241.2 $\pm$ 8.43
Confidentiality	2	120	54.02 $\pm$ 4.87	301.9 $\pm$ 7.89	12.27 $\pm$ 0.95	247.9 $\pm$ 8.32

Table 7. Blue agent performance across chokepoint levels per security objective. Quantifies the defensive benefit (or lack thereof) each strategy derives from topology chokepoints. Values are mean $\pm$ SE.

Objective	Blue	Chokepoints	N	Cum. Infected	Root Shells	TP Restore
Availability	BlueMonitor	0	60	329.3 $\pm$ 1.09	248.4 $\pm$ 1.27	0.00 $\pm$ 0.00
Availability	BlueMonitor	1	60	58.33 $\pm$ 7.31	45.50 $\pm$ 5.87	0.00 $\pm$ 0.00
Availability	BlueMonitor	2	60	52.42 $\pm$ 7.60	40.92 $\pm$ 6.08	0.00 $\pm$ 0.00
Availability	ReactAnalyse	0	60	329.2 $\pm$ 1.08	247.7 $\pm$ 1.18	0.00 $\pm$ 0.00
Availability	ReactAnalyse	1	60	59.42 $\pm$ 7.50	47.42 $\pm$ 6.24	0.00 $\pm$ 0.00
Availability	ReactAnalyse	2	60	54.33 $\pm$ 7.43	42.17 $\pm$ 5.75	0.00 $\pm$ 0.00
Availability	ReactDecoy	0	60	329.2 $\pm$ 1.10	248.7 $\pm$ 1.08	0.00 $\pm$ 0.00
Availability	ReactDecoy	1	60	58.55 $\pm$ 7.57	46.13 $\pm$ 6.04	0.00 $\pm$ 0.00
Availability	ReactDecoy	2	60	52.97 $\pm$ 7.28	42.15 $\pm$ 5.89	0.00 $\pm$ 0.00
Availability	ReactRemove	0	60	328.8 $\pm$ 1.07	247.4 $\pm$ 1.03	0.00 $\pm$ 0.00
Availability	ReactRemove	1	60	67.92 $\pm$ 7.98	47.58 $\pm$ 6.09	0.00 $\pm$ 0.00
Availability	ReactRemove	2	60	57.92 $\pm$ 7.38	42.75 $\pm$ 6.08	0.00 $\pm$ 0.00
Availability	ReactRestore	0	60	465.7 $\pm$ 1.99	8.08 $\pm$ 0.64	392.5 $\pm$ 6.02
Availability	ReactRestore	1	60	224.6 $\pm$ 24.07	34.58 $\pm$ 3.89	183.0 $\pm$ 19.54
Availability	ReactRestore	2	60	209.8 $\pm$ 24.18	32.50 $\pm$ 3.76	171.0 $\pm$ 20.09
Integrity	BlueMonitor	0	60	321.2 $\pm$ 5.65	242.0 $\pm$ 4.32	0.00 $\pm$ 0.00
Integrity	BlueMonitor	1	60	124.0 $\pm$ 21.12	48.33 $\pm$ 6.32	0.00 $\pm$ 0.00
Integrity	BlueMonitor	2	60	170.6 $\pm$ 25.25	42.32 $\pm$ 5.89	0.00 $\pm$ 0.00
Integrity	ReactAnalyse	0	60	326.8 $\pm$ 1.73	244.2 $\pm$ 1.64	0.00 $\pm$ 0.00
Integrity	ReactAnalyse	1	60	126.9 $\pm$ 21.07	49.62 $\pm$ 6.43	0.00 $\pm$ 0.00
Integrity	ReactAnalyse	2	60	154.7 $\pm$ 24.18	43.88 $\pm$ 6.04	0.00 $\pm$ 0.00
Integrity	ReactDecoy	0	60	326.8 $\pm$ 1.77	244.3 $\pm$ 1.64	0.00 $\pm$ 0.00
Integrity	ReactDecoy	1	60	121.8 $\pm$ 20.48	47.62 $\pm$ 6.40	0.00 $\pm$ 0.00
Integrity	ReactDecoy	2	60	117.6 $\pm$ 20.68	42.48 $\pm$ 5.99	0.00 $\pm$ 0.00
Integrity	ReactRemove	0	60	326.4 $\pm$ 1.72	245.9 $\pm$ 1.70	0.00 $\pm$ 0.00
Integrity	ReactRemove	1	60	144.7 $\pm$ 21.74	50.48 $\pm$ 6.68	0.00 $\pm$ 0.00
Integrity	ReactRemove	2	60	161.7 $\pm$ 23.95	46.00 $\pm$ 6.46	0.00 $\pm$ 0.00
Integrity	ReactRestore	0	60	451.9 $\pm$ 7.87	8.43 $\pm$ 0.53	369.8 $\pm$ 8.48
Integrity	ReactRestore	1	60	284.3 $\pm$ 24.11	33.33 $\pm$ 2.88	199.7 $\pm$ 19.45
Integrity	ReactRestore	2	60	290.4 $\pm$ 25.50	31.58 $\pm$ 3.10	179.3 $\pm$ 19.46
Confidentiality	BlueMonitor	0	30	659.3 $\pm$ 60.29	115.8 $\pm$ 19.75	0.00 $\pm$ 0.00
Confidentiality	BlueMonitor	1	30	254.2 $\pm$ 10.49	64.17 $\pm$ 10.88	0.00 $\pm$ 0.00
Confidentiality	BlueMonitor	2	30	255.8 $\pm$ 9.35	59.50 $\pm$ 10.98	0.00 $\pm$ 0.00
Confidentiality	ReactAnalyse	0	30	659.7 $\pm$ 60.30	115.2 $\pm$ 19.56	0.00 $\pm$ 0.00
Confidentiality	ReactDecoy	0	30	659.6 $\pm$ 60.35	115.4 $\pm$ 19.71	0.00 $\pm$ 0.00
Confidentiality	ReactDecoy	1	30	262.0 $\pm$ 9.24	62.17 $\pm$ 10.97	0.00 $\pm$ 0.00
Confidentiality	ReactDecoy	2	30	270.7 $\pm$ 9.96	60.10 $\pm$ 11.18	0.00 $\pm$ 0.00
Confidentiality	ReactRemove	0	30	658.7 $\pm$ 59.99	117.2 $\pm$ 20.01	0.00 $\pm$ 0.00
Confidentiality	ReactRemove	1	30	263.3 $\pm$ 10.85	117.8 $\pm$ 15.15	0.00 $\pm$ 0.00
Confidentiality	ReactRemove	2	30	283.7 $\pm$ 9.81	113.2 $\pm$ 15.30	0.00 $\pm$ 0.00
Confidentiality	ReactRestore	0	30	697.7 $\pm$ 42.85	8.50 $\pm$ 0.72	222.7 $\pm$ 31.09
Confidentiality	ReactRestore	1	30	409.5 $\pm$ 16.03	36.17 $\pm$ 3.75	281.3 $\pm$ 26.06
Confidentiality	ReactRestore	2	30	397.5 $\pm$ 17.45	36.33 $\pm$ 4.03	290.8 $\pm$ 26.75

Table 8. Effect of topology size ($h=s$) on metrics per security objective. Values are mean $\pm$ SE.

Objective	$h=s$	N	TP Attacks	Unique Compr.	Cum. Infected	Root Shells	TP Restore
Availability	5	300	124.2 $\pm$ 9.27	6.99 $\pm$ 0.14	128.0 $\pm$ 9.27	71.10 $\pm$ 6.09	27.02 $\pm$ 5.14
Availability	10	300	152.1 $\pm$ 9.02	19.16 $\pm$ 0.56	157.7 $\pm$ 8.92	81.30 $\pm$ 5.74	46.60 $\pm$ 6.72
Availability	15	300	233.2 $\pm$ 8.58	68.89 $\pm$ 2.75	250.0 $\pm$ 8.23	132.0 $\pm$ 4.70	75.68 $\pm$ 8.77
Integrity	5	300	127.3 $\pm$ 8.91	9.89 $\pm$ 0.33	191.2 $\pm$ 10.87	71.42 $\pm$ 5.79	30.07 $\pm$ 5.21
Integrity	10	300	177.6 $\pm$ 9.24	30.68 $\pm$ 1.39	228.3 $\pm$ 10.16	94.00 $\pm$ 5.51	58.62 $\pm$ 7.55
Integrity	15	300	206.7 $\pm$ 8.81	57.06 $\pm$ 3.21	270.5 $\pm$ 9.25	118.7 $\pm$ 5.38	61.07 $\pm$ 7.83
Confidentiality	5	130	144.9 $\pm$ 12.57	9.51 $\pm$ 0.37	419.9 $\pm$ 25.31	51.84 $\pm$ 6.72	44.54 $\pm$ 9.93
Confidentiality	10	130	171.9 $\pm$ 12.77	27.26 $\pm$ 1.48	431.4 $\pm$ 24.16	69.84 $\pm$ 6.83	58.62 $\pm$ 11.07
Confidentiality	15	130	225.2 $\pm$ 13.76	76.37 $\pm$ 5.55	471.3 $\pm$ 21.69	114.1 $\pm$ 7.50	80.27 $\pm$ 13.82

References

1. A2Labs-Research: MATRIX. <https://github.com/A2Labs-Research/MATRIX> (2026)
2. Applebaum, A., Dennler, C., Dwyer, P., Moskowitz, M., Nguyen, H., Nichols, N., Park, N., Rachwalski, P., Rau, F., Webster, A., et al.: Bridging automated to autonomous cyber defense: Foundational analysis of tabular Q-learning. In: Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security. pp. 149–159 (2022)
3. Autonomous-Resilient-Cyber-Defence: PrimAITE. <https://github.com/Autonomous-Resilient-Cyber-Defence/PrimAITE> (2025)
4. Bates, E., Hicks, C., Mavroudis, V.: Beyond rewards in reinforcement learning for cyber defence. arXiv preprint arXiv:2602.04809 (2026)
5. Cadet, X., Boboila, S., Dharmawan, S.H., Oprea, A., Chin, P.: PoolFlip: A multi-agent reinforcement learning security environment for cyber defense. In: International Conference on Game Theory and AI for Security. pp. 172–192. Springer (2025)
6. Cadet, X., Boboila, S., Koh, E., Chin, P., Oprea, A.: Quantitative resilience modeling for autonomous cyber defense. In: Reinforcement Learning Conference (2025), <https://openreview.net/forum?id=wgz2rpXptd>
7. Foley, M., Hicks, C., Highnam, K., Mavroudis, V.: Autonomous network defence using reinforcement learning. In: Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security. pp. 1252–1254 (2022)
8. Gueye, A., Mell, P., Harang, R., La, R.J.: Defensive resource allocations with security chokepoints in IPv6 networks. In: IFIP Annual Conference on Data and Applications Security and Privacy. pp. 261–276. Springer (2015)
9. Hammar, K., Li, T., Stadler, R., Zhu, Q.: Adaptive security response strategies through conjectural online learning. IEEE Transactions on Information Forensics and Security (2025)
10. Hammar, K., Stadler, R.: Learning near-optimal intrusion responses against dynamic attackers. IEEE Transactions on Network and Service Management **21**(1), 1158–1177 (2023)
11. Hicks, C., Bates, E., McFadden, S., Thompson, I.S., Foley, M., Chapman, E., Dice, N.E., Samaddar, A., Sylvester, J., Neema, H., et al.: Building better environments for autonomous cyber defence. arXiv preprint arXiv:2604.08805 (2026)
12. Janisch, J., Pevný, T., Lisý, V.: NASimEmu: Network attack simulator & emulator for training agents generalizing to novel scenarios. In: European Symposium on Research in Computer Security. pp. 589–608. Springer (2023)

13. Khurana, M., Jain, R.: SoK: Measuring what matters for closed-loop security agents. arXiv preprint arXiv:2510.01654 (2025)
14. Kiely, M., Ahiskali, M., Borde, E., Bowman, B., Bowman, D., Van Bruggen, D., Cowan, K., Dasgupta, P., Devendorf, E., Edwards, B., et al.: CAGE challenge 4: A scalable multi-agent reinforcement learning gym for autonomous cyber defence. *AI Magazine* **46**(3), e70021 (2025)
15. Kiely, M., Bowman, D., Standen, M., Moir, C.: On autonomous agents in a cyber defence environment. arXiv preprint arXiv:2309.07388 (2023)
16. Kunz, T., Fisher, C., La Novara-Gsell, J., Nguyen, C., Li, L.: A multiagent CyberBattleSim for RL cyber operation agents. In: 2022 International Conference on Computational Science and Computational Intelligence (CSCI). pp. 897–903. IEEE (2022)
17. Lamichhane, P.B., Hong, L., Shetty, S.: A quantitative risk analysis model and simulation of enterprise networks. In: 2018 IEEE 9th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON). pp. 844–850. IEEE (2018)
18. Landolt, C.R., Würsch, C., Meier, R., Mermoud, A., Jang-Jaccard, J.: Multi-agent reinforcement learning in cybersecurity: From fundamentals to applications. arXiv preprint arXiv:2505.19837 (2025)
19. Le, A.T., Epiphaniou, G., Maple, C., Kyriakopoulos, K.G., Kiarie, L., Aristodemou, M., Phillips, I.: Automated APT defense using reinforcement learning and attack graph risk-based situation awareness. In: Proceedings of the Workshop on Autonomous Cybersecurity. pp. 23–33 (2023)
20. Le, D.H., Stadler, R.: Learning optimal defender strategies for CAGE-2 using a POMDP model. In: 2025 21st International Conference on Network and Service Management (CNSM). pp. 1–9. IEEE (2025)
21. Li, L., El Rami, J.P.S., Taylor, A., Rao, J.H., Kunz, T.: Unified emulation-simulation training environment for autonomous cyber agents. In: International Conference on Machine Learning for Networking. pp. 130–144. Springer (2022)
22. Mayoral-Vilches, V., Sanz-Gómez, M., Balassone, F., De Torres, M.D.M., Nicolaou, G., Borines, S.R., Graziano, A., Zabalegui, P., Gil-Uriarte, E.: Dynamic cyber ranges. arXiv preprint arXiv:2604.24184 (2026)
23. Microsoft Defender Research Team: CyberBattleSim. <https://github.com/microsoft/cyberbattlesim> (2021), created by Christian Seifert, Michael Betser, William Blum, James Bono, Kate Farris, Emily Goren, Justin Grana, Kristian Holsheimer, Brandon Marken, Joshua Neil, Nicole Nichols, Jugal Parikh, Haoran Wei.
24. Miehl, E., Rasouli, M., Teneketzis, D.: A POMDP approach to the dynamic defense of large-scale cyber networks. *IEEE Transactions on Information Forensics and Security* **13**(10), 2490–2505 (2018)
25. Morris, A., Procter, R., Wallbank, C.: Evaluating reinforcement learning agents for autonomous cyber defence. *Applied AI Letters* **6**(3), e125 (2025)
26. Nagaraja, S.: Topology of covert conflict: (transcript of discussion). In: International Workshop on Security Protocols. pp. 329–332. Springer (2005)
27. Nguyen, A.T., Hertzberg, A., Teixeira, A.M.: Centrality-based security allocation in networked control systems. In: International Conference on Critical Information Infrastructures Security. pp. 212–230. Springer (2024)
28. Nieves, M., Dempsey, K., Pillitteri, V.Y., et al.: An introduction to information security. NIST special publication **800**(12), 101 (2017)

29. Nwodo, K., Van Roy, J., Hnedzko, D., Stavrou, A.: Enter the MATRIX: Learning & validating cybersecurity strategies in hybrid simulation & emulation environments. In: 2026 IEEE Conference on Dependable and Secure Computing (DSC). pp. 1–8. IEEE (2026)
30. Pal, R., Bluestein, L., Askerbekov, T., Siegel, M.: A formal and deployable gaming operation to defend IT/OT networks. In: 2025 Winter Simulation Conference (WSC). pp. 1943–1954. IEEE (2025)
31. Ramamurthy, A., Dhir, N.: General autonomous cybersecurity defense: Learning robust policies for dynamic topologies and diverse attackers. arXiv preprint arXiv:2506.22706 (2025)
32. Standen, M., et al.: Cyber operations research gym. <https://github.com/cage-challenge/CybORG> (2022), created by Maxwell Standen, David Bowman, Son Hoang, Toby Richer, Martin Lucas, Richard Van Tassel, Phillip Vu, Mitchell Kiely, KC C., Natalie Konschnik, Joshua Collyer
33. Standen, M., Lucas, M., Bowman, D., Richer, T.J., Kim, J., Marriott, D.: CybORG: A gym for the development of autonomous cyber agents. arXiv preprint arXiv:2108.09118 (2021)
34. Symes Thompson, I., Caron, A., Hicks, C., Mavroudis, V.: Entity-based reinforcement learning for autonomous cyber defence. In: Proceedings of the Workshop on Autonomous Cybersecurity. pp. 56–67 (2023)
35. ThreatNG Security: Choke Point. <https://www.threatngsecurity.com/glossary/choke-point>, last accessed 2026/08/30
36. Vyas, S., Mavroudis, V., Burnap, P.: Towards the deployment of realistic autonomous cyber network defence: A systematic review. ACM Computing Surveys **58**(1), 1–36 (2025)
37. Zhang, L., Zhu, T., Hussain, F.K., Ye, D., Zhou, W.: A game-theoretic method for defending against advanced persistent threats in cyber systems. IEEE Transactions on Information Forensics and Security **18**, 1349–1364 (2022)
38. Zhong, W., Jiang, Y., Yang, Q., Zhuansun, C., Sun, Y., Su, S., Tian, Z., Liu, Y.: Node betweenness centrality based defense strategy against topology inference threat: A static game theoretical perspective. In: 2024 IEEE Cyber Science and Technology Congress (CyberSciTech). pp. 52–58. IEEE (2024)